

American International University-Bangladesh (AIUB)

Faculty of Science and Technology

Department of Computer Science

Web Technologies

CSE 3215

PROJECT REPORT

Project Title:

E-Commerce Store (ShopNest)

Semester: [Spring 2025–2026]

Submission Date: [05, 17, 2026]

Section: [c]

Submitted To:

Rifat Al Mamun Rudro

Lecturer, Department of Computer Science, Faculty of Science and Technology
American International University-Bangladesh

Group Information

Fill in all fields below. Each group member must sign before submission.

Group Number: 04

Project Title: E-Commerce Store (ShopNest)

Student Information:

Full Name:MD.ABIDUL ISLAM

Student ID:22-47875-2

Program: CSE

Email: abidjihan1123@gmail.com

Signature: abid

Declaration: We hereby declare that this project report is our original work. Any external resources or references used have been properly cited. We have not engaged in any form of academic dishonesty.

Marks Sheet

To be filled by the instructor only. Students must NOT write in this section.

Group No.		Section	
Member 1 ID			

C03: Illustrate a Multi-Tier Web Application for the Targeted Society					
Feature Implementation (0-3)			Feature Implementation (0-3)		Completeness (0-3)
Basic Web Security	UI (HTML/CSS)	Feature Completeness	DB	Auth (Session/cookie)	MVC
C04: Coordinate with Group Members to Develop Multi-Tier Application					
Technical Knowledge (0-3)			Collaborative Teamwork (0-3)	Communication/Promptness (0-3)	
JS validation	PHP Validation	Ajax/JSON	Git Contribution	VIVA	
Expected Outcome (0-2)		C03 Total	C04 Total	Project Total	

Evaluated By: Rifat Al Mamun Rudro **Date:** 18th May 2025

Course Teacher Signature:

Chapter 1: Project Overview

1.1 Project Description

[Task 3 : Customer Shopping Cart & Checkout]

Description: Our web application is an e-commerce store where customers can view available products, create and remove items from shopping cart and purchase them online. The manual sale of the product becomes a thing of the past because with it gives us an easy and seamless digital shopping process.

My role for the application is task 3 — Shopping Cart & Checkout. This explains the purchase flow which includes end to end customer journey. Users can view a grid of products for sale, search by keyword and filter by category — all without reloading the page! The products can be added on a session-based cart with updated quantities, & removal of items using AJAX calls that refresh totals in real-time. Then customers can checkout, use a saved address to ship their order, select a payment method and place an new order which is stored in the database and the cart gets emptied automatically.

Targeting general online consumers who want to quickly search for a product, manage their cart and checkout without the additional steps.

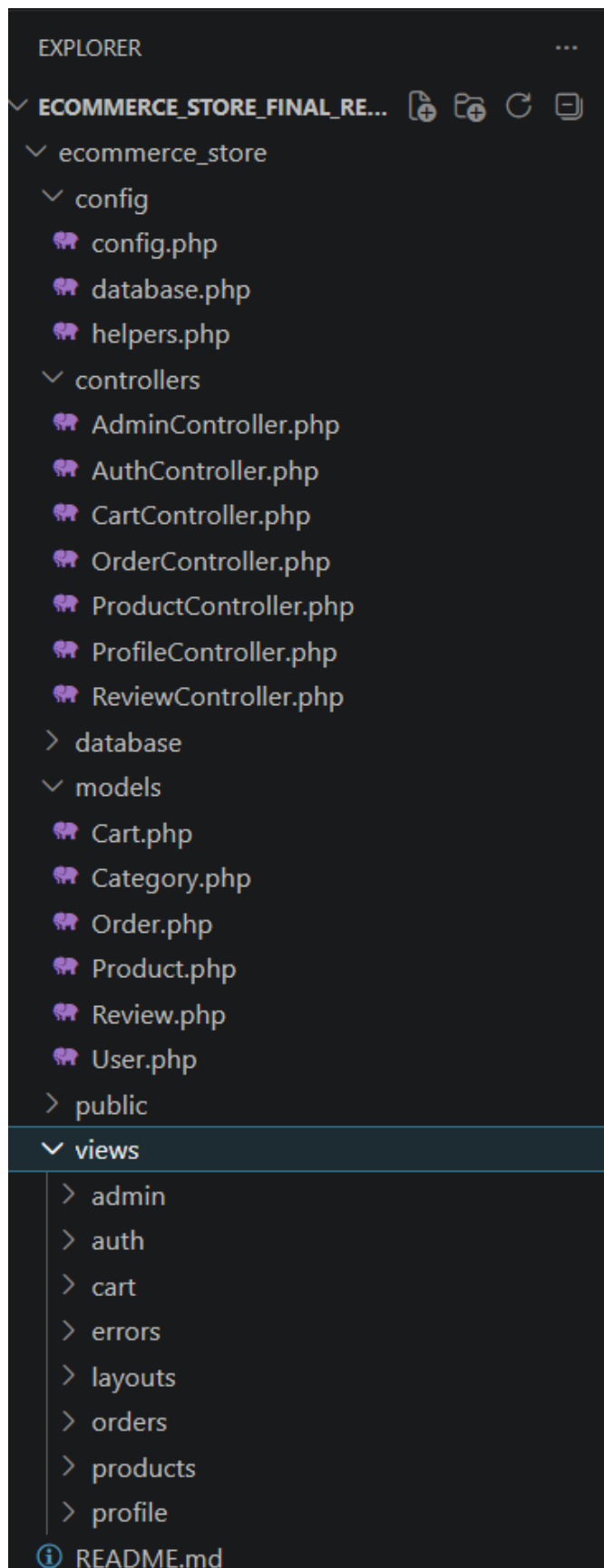
The feature is written in PHP (MVC structured), PDO with prepared statements for all database queries and AJAX finter for faster and responsive user experience.]

1.2 Technologies Used

List all technologies, frameworks, and tools used in this project.

Layer	Technology / Tool	Purpose
Frontend	HTML5, CSS3, Bootstrap	UI & Styling
	[e.g., JavaScript / jQuery]	[e.g., Client-side validation, AJAX]
Backend	[e.g., PHP]	[e.g., Server-side logic]
Database	[e.g., MySQL]	[e.g., Data storage]
Auth	[e.g., PHP Sessions / Cookies]	[e.g., Login management]
Version Control	Git / GitHub	Collaboration & versioning
Server	[e.g., XAMPP / Apache]	[e.g., Local development]

1.3 System Architecture / MVC Diagram



1.4 Database Schema

List your database tables and their key fields.

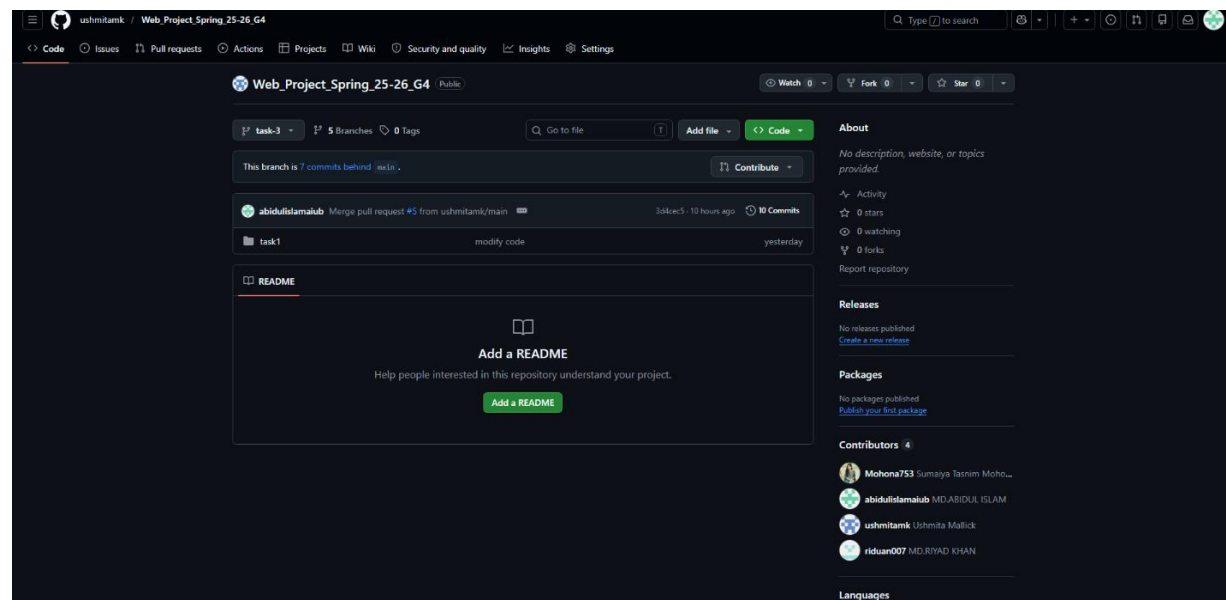
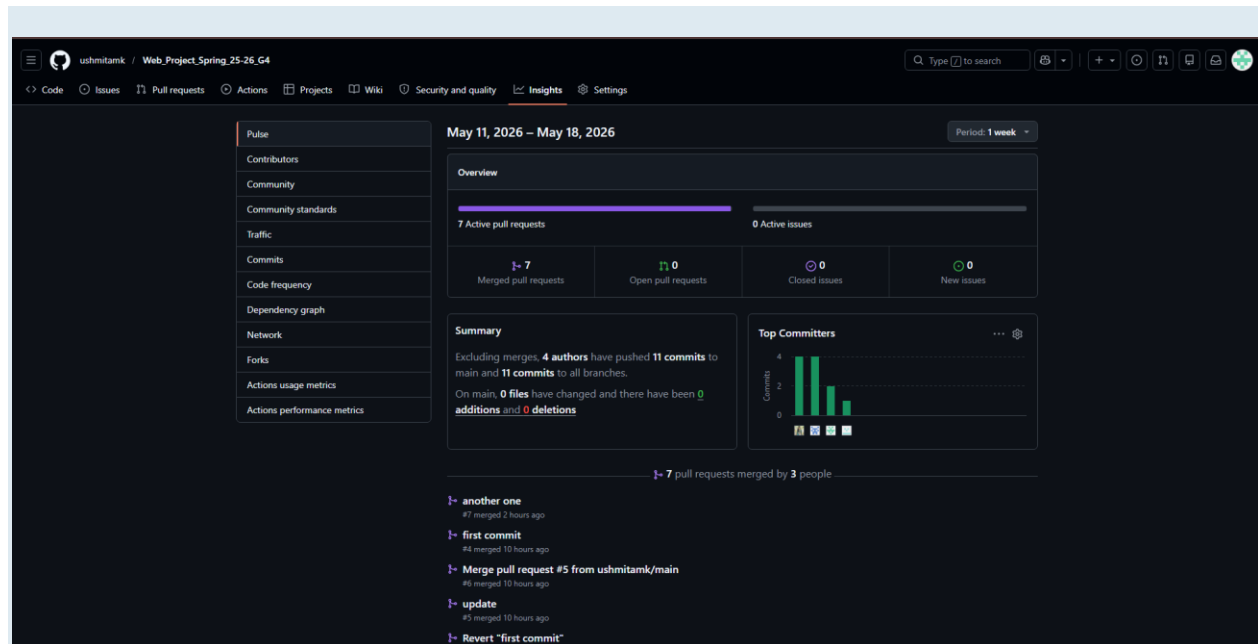
Table Name	Field Name	Data Type	Description
[products]	[id]	[INT]	[Primary Key]
	[category_id]	[INT]	[Foreign]
	name	[VARCHAR(255)]	Product name
	description	TEXT	
	price	DECIMAL(10,2)	
	stock_qty	INT	
	primary_image_path	VARCHAR(255)	
	is_available	TINYINT(1)	
	created_at	DATETIME	
categories	id	INT	Primary Key
	parent_id	INT	Foreign Key
	name	VARCHAR(120)	
orders	id	INT	Primary Key
	user_id	INT	Foreign Key
	shipping_address	TEXT	
	payment_method	ENUM	
	total_amount	DECIMAL(10,2)	
	status	ENUM	
	created_at	DATETIME	
order_items	id	INT	Primary Key
	order_id	INT	Foreign Key
	product_id	INT	Foreign Key
	quantity	INT	
	unit_price	DECIMAL(10,2)	

Chapter 2: Git Contribution & Task Allocation

2.1 GitHub Repository Link

Repository URL: https://github.com/ushmitamk/Web_Project_Spring_25-26_G4

2.2 Git Contribution Screenshot



Chapter 3: Features, Source Code & Screenshots

Feature :1 [Product Catalogue with AJAX Search & Category Filter]

Description :This feature displays a grid of all available products to customers. It includes a live keyword search bar and a category filter dropdown. Both work via AJAX — meaning the product grid updates instantly without reloading the page. Customers can also click any product to open its detail page showing the image, description, price, stock status, and average rating.

Code:

```
ProductController.php X
C:\Users\ASUS\Downloads\ecommerce_store_final_ready\ecommerce_store\controllers> ProductController.php
1 <?php
2 class ProductController
3 {
4     public function index()
5     {
6         $filters = [
7             'q' => trim($_GET['q'] ?? ''),
8             'category_id' => isset($_GET['category_id']) ? (int)$_GET['category_id'] : null,
9         ];
10        $products = Product::available($filters);
11        $categories = Category::hierarchyOptions();
12        render('products/index', ['products' => $products, 'categories' => $categories, 'filters' => $filters]);
13    }
14
15    public function detail($id)
16    {
17        $product = Product::find($id);
18        if (!$product) {
19            http_response_code(404);
20            render('errors/404');
21            return;
22        }
23        $canReview = false;
24        $hasReviewed = false;
25        if (is_logged_in() && current_role() === 'customer') {
26            $canReview = Cart::hasProduct($id) || Order::userHasPurchasedProduct(current_user_id(), $id);
27            $hasReviewed = Review::exists($id, current_user_id());
```



```

<?php
class Product
{
    private static function ratingJoinSql()
    {
        return "LEFT JOIN (
            SELECT product_id, ROUND(AVG(rating), 1) AS avg_rating, COUNT(*) AS review_count
            FROM reviews
            GROUP BY product_id
        ) rv ON rv.product_id = p.id";
    }

    private static function ratingSelectSql()
    {
        return "COALESCE(rv.avg_rating, 0) AS avg_rating, COALESCE(rv.review_count, 0) AS review_count";
    }

    public static function available($filters = [])
    {
        $params = [];
        $where = ["p.is_available = 1"];
        if (!empty($filters['q'])) {
            $where[] = "(p.name LIKE ? OR p.description LIKE ?)";
            $params[] = '%' . $filters['q'] . '%';
            $params[] = '%' . $filters['q'] . '%';
        }
    }
}

```

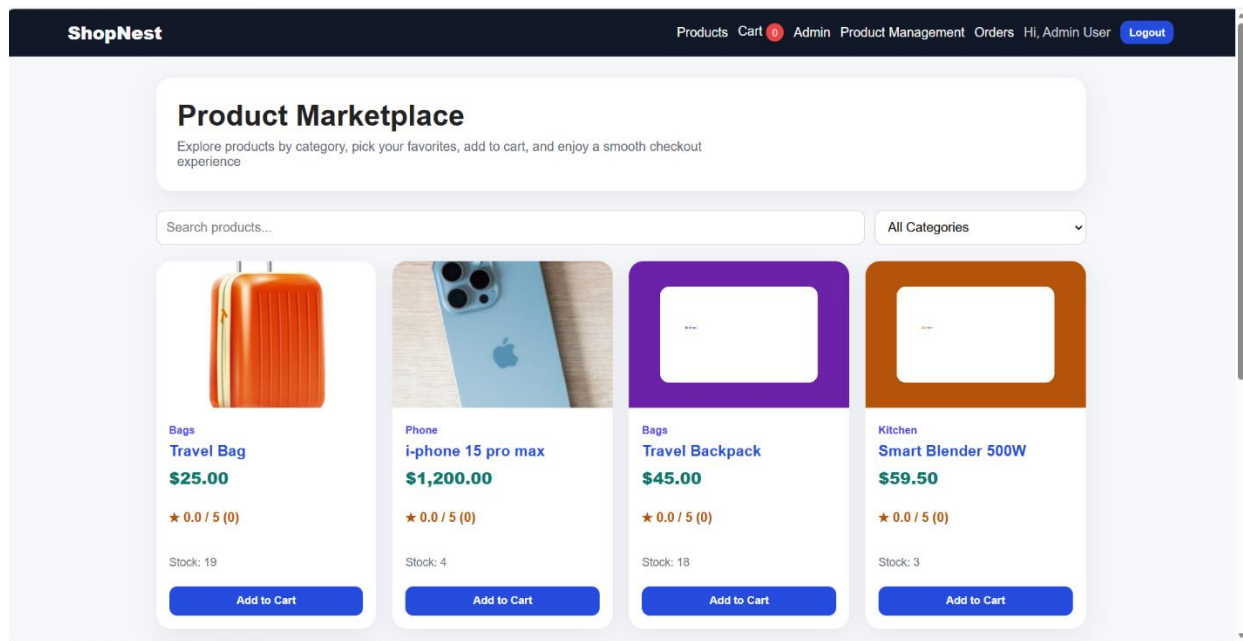
Code explanation

When a customer opens the product catalogue page, the first thing that runs is the `index()` function. It takes two values from the URL — i.e. the search keyword entered by customer and category they selected. It calls `Product::available()` with both values so that products are fetched from the database. This also loads up the dropdown list of categories. Ultimately it sends everything to the product page for display.

You can also notice that `apiProducts()` is called without any comments every time the customer writes in search box or changes a category filter. It returns a JSON response with our product grid in html and does not reload the entire page. The old product grid is replaced by JavaScript on the fly with the new one.

The `available()` function constructs the query for the database incrementally. It starts with filtering the products so that only those which have available in stock are visible. It adds a condition that searches for the word inside the product name and the product description if you provide a search keyword. If a category has been selected it also include for additional condition that show the products based on that category only. It connects to the reviews table, and computes the average (star) ratings per product as well. It then returns all the matched products and displays them as cards on the catalogue page.

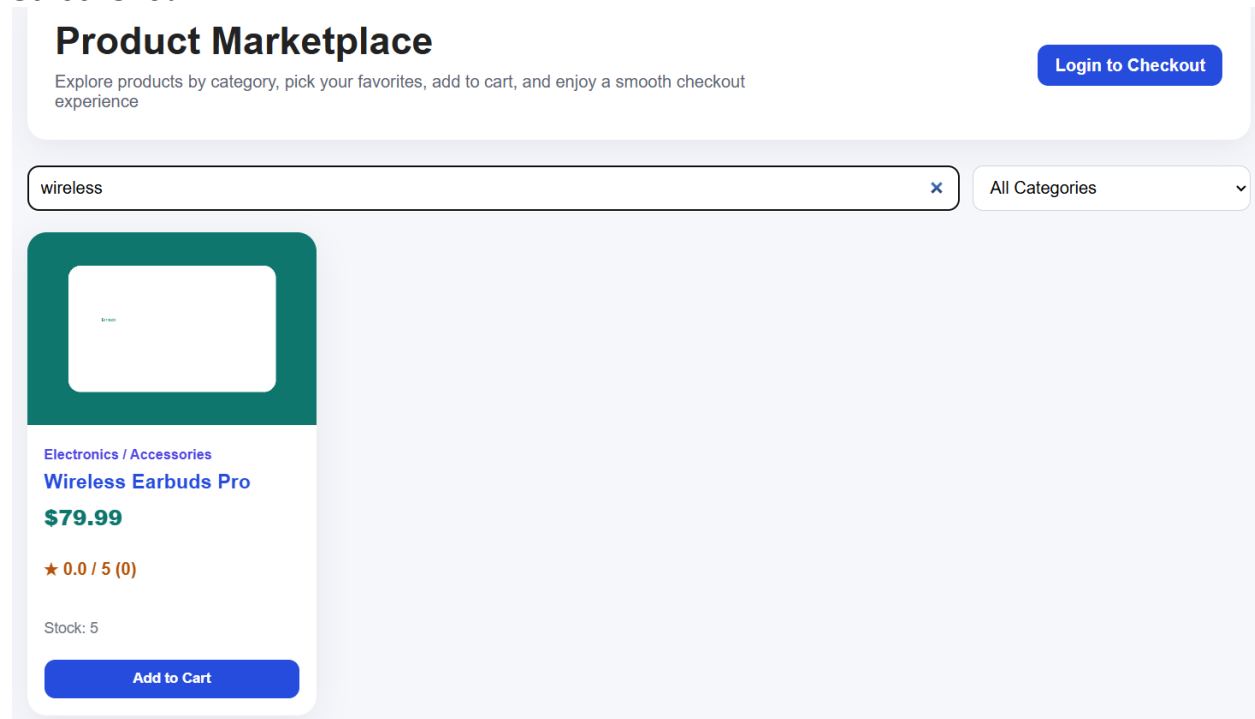
Screenshot:



Feature :2[Add to Cart (AJAX)]

Description: Customers can add any available product to their cart by clicking the "Add to Cart" button. The request is sent via AJAX to avoid a page reload. The cart is stored in `$_SESSION['cart']` as `[product_id => quantity]`. The quantity is capped at the available stock. The navbar cart count updates instantly via DOM manipulation.

Screenshot:



Feature :3 [Cart Page with Quantity Update & Remove (AJAX)]

Description : The cart page lists all items in the session cart with their name, unit price, quantity controls (+ / -), line total, and a remove button. Each button sends an AJAX call to update or remove items. Only the totals section updates on the page — no full reload happens.

Code:

```
<?php
class CartController
{
    public function index()
    {
        $items = Cart::items();
        $reviewed = [];
        if (is_logged_in() && current_role() === 'customer') {
            foreach ($items as $item) {
                $reviewed[(int)$item['id']] = Review::exists($item['id'], current_user_id());
            }
        }
        render('cart/index', ['items' => $items, 'total' => Cart::total(), 'reviewed' => $reviewed]);
    }

    public function apiAdd()
    {
        $data = $_POST ? input_json():
        $productId = isset($data['product_id']) ? (int)$data['product_id'] : 0;
        if ($productId <= 0) json_response(['ok' => false, 'message' => 'Invalid product.'], 422);
        $result = Cart::add($productId);
        json_response($result, $result['ok'] ? 200 : 422);
    }

    public function apiUpdate()
    {
        $data = $_POST ? input_json():
```

Code Explanation

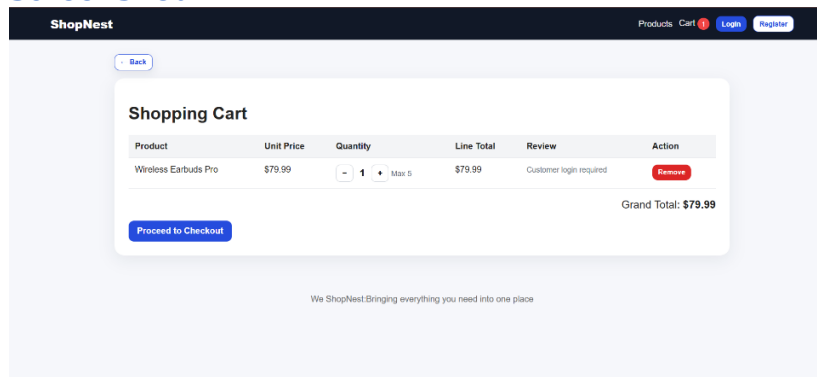
This portion of code is responsible for updating the quantity as well as removing items from the cart depending on whether customer clicks plus, minus or remove button.

The function `apiUpdate()` is called when the customer clicks on plus or minus button of a cart item. Firstly, it reads the product id and the new quantity sent from the browser. It checks the validity of the product ID — if it fails then it returns an error response right away. If there's nothing wrong it calls `Cart::update()` with both values to do the actual update. Finally, the result is returned to the browser as a JSON response so the JavaScript can update both the line total and grand total on page without requiring a reload.

The `apiRemove()` method is run when a customer clicks the Remove button of one of the cart items. It read the product id from request and checks for it's validity. If they are valid it calls `Cart::remove()` to remove that item from the

cart in session. We then return the updated grand total and cart count to browser, which means that JavaScript can update these on the fly once again. Real work of updating quantity has done by `Cart::update()` method. The first does a lookup in the database to see whether that product exists or not. It then ensures that the new quantity is not less than 0 (using `max()`) and. If the quantity is set to zero the product will be fully removed from the session cart. If not, the new amount is stored in `$_SESSION['cart']`. It then sends back an updated quantity, line total, grand total and cart count to the browser. The `Cart::remove()` method just uses `unset()`, to remove the product from `$_SESSION['cart']`. After that it simply returns the updated grand total and again cart count so that JavaScript can apply entered process and reflect on the Top of page as soon as possible means without reloading your page.

Screenshot:



Feature :4 [Checkout & Order Confirmation]

Description The checkout page shows the cart summary, saved shipping addresses as radio buttons (pre-filled from the customer's profile), and a payment method selection (Cash or Card). On submission, PHP validates the cart, checks stock, writes the order to the database, decrements stock, clears the session cart, and redirects to a confirmation page showing the order ID and summary.

Code:

```
{
  {
    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
      if ($address === '' ) $errors['shipping_address'] = 'Shipping address is required. ';
      if (!$errors) {
        try {
          $orderId = Order::createFromCart(current_user_id(), $address, $payment);
          redirect('/confirmation/' . $orderId);
        } catch (Exception $e) {
          $errors['cart'] = $e->getMessage();
        }
      }
    }
    render('cart/checkout', ['items' => $items, 'total' => Cart::total(), 'addresses' => $addresses, 'errors' => $errors]);
  }

  public function confirmation($id)
  {
    require_customer();
    $order = Order::findForUser($id, current_user_id());
    if (!$order) {
      http_response_code(404);
      render('errors/404');
      return;
    }
    render('cart/confirmation', ['order' => $order, 'items' => Order::itemsForOrder($id)]);
  }
}
```

Code Explanation

This part addresses checkout and order confirmation, the part of a system where a customer places an order.

The checkout() function is executed when the customer opens checkout. First it calls require_customer() to check that only logged in customers can view this page — if they're not logged on they are redirected to the login page. Then it loads all the items currently in cart, and load up customer saved shipping addresses from the profile to be displayed as radio button on checkout form.

On clicking the Place Order button, validation of everything inside the function starts. Firstly, check if the cart is not empty. Then, it validates that you chose a proper payment method i.e. Cash or Card It checks whether the user provided a shipping address — either by selecting one from the radio buttons or entering a new one in the text box. If any of these checks fail, an error message is printed directly under the corresponding field on page without a page refresh.

The function calls Order::createFromCart() to create the order on the database, if all validations pass. Once the order is done successfully, the customer will be redirected to confirmation page with order ID and summary.

The place to securely save the order to the database is in the createFromCart() function. It runs a DB transaction => Anything that fails will rollback all changes. It then iterates through each item in the basket and goes totally kskks even though its been checked multiple times in the past up until that point, at the moment of ordering it checks if there is enough stock left for each order. If any of the products do not have sufficient stock, an error is thrown meaning that the transaction will be cancelled.

If all the stock checks pass it simply inserts a single row into the orders table It stores the customer ID, shipping address (including delivery and billing) payment method and total

amount. It will then iterate through the cart items again and insert a single row for each into the order items table. This calls `Product::decrementStock()` after each item is inserted to decrease the stock quantity of that product in the database.

When everything is saved successfully, transaction commits to make all changes permanent. `Cart::clear()` is then executed to clear the session cart completely and the new order ID returned so that customer will be redirected to the confirmation page with their order details.

Screenshot:

ShopNest Products Cart **1** My Orders Profile Hi, abidul Logout

[← Back](#)

Checkout

Product	Qty	Unit Price	Total
Wireless Earbuds Pro	1	\$79.99	\$79.99

Total: **\$79.99**

Shipping Address

☒ Use a new address

New shipping address

Payment Method

☐ Cash ☐ Card

Place Order

Chapter 5: Conclusion & Future Work

5.1 Summary

This project was to develop the full customer journey for an e-commerce web application in PHP, built on the MVC pattern. In Task 3 the goal was to show a way of allowing customers to browse products, search for and filter products and manage a session scoped cart as well as checkout. All the 6 features were implemented and the working version is ready. You can check product catalogue, view details of a product, put products in cart, increase/decrease quantity of item, remove a particular item and order it with saved shipping address & payment option. The application is also fully AJAX-based, so the whole experience felt very seamless and responsive with little to zero page refreshes. There is no separate database table for the cart that survives server restarts – everything goes completely into session, and then we store orders in a transaction (if anything fails during order validation, nothing will be saved on DB)

5.2 Challenges Faced

One of the biggest challenges was implementing the cart system in AJAX correctly. To ensure that the count of items in the cart was updated immediately after every add, update and remove action, I needed to communicate between PHP (backend) and JavaScript (frontend).

But there were another few challenges like the stock getting validated at checkout. Not just checking the stock when adding to cart, because still there was possibility of 1 or more items already being sold out between stash time and the moment the order was submitted.

`Order::createFromCart()` solved this problem by having the product stock refetch inside a database transaction.

All four students operated off the same respective tables, and so using a shared database became tricky as well. Any schema or data handled incorrectly could break other teammates' features. This was handled well by just adhering to the shared schema and not modifying tables.

5.3 Future Improvements

- Adding a wishlist feature so customers can save products they are interested in but not ready to purchase yet.
- Implementing a coupon and discount code system at checkout so customers can apply promotional codes to reduce their order total.
- Adding real-time stock notifications so customers are automatically alerted when an out-of-stock product becomes available again.
- Improving the checkout process by integrating a real payment gateway such as Stripe or PayPal instead of just Cash or Card selection.

References

- [1] The PHP Group. (2024). PHP Manual — Sessions. <https://www.php.net/manual/en/book.session.php>
- [2] The PHP Group. (2024). PHP Manual — PDO (PHP Data Objects). <https://www.php.net/manual/en/book.pdo.php>
- [3] Mozilla Developer Network. (2024). Fetch API. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- [4] Mozilla Developer Network. (2024). XMLHttpRequest — AJAX Guide. <https://developer.mozilla.org/en-US/docs/Web/Guide/AJAX>
- [5] W3Schools. (2024). PHP MVC Architecture. <https://www.w3schools.com/php/>
- [6] Bootstrap Team. (2024). Bootstrap 5 Documentation. <https://getbootstrap.com/docs/5.0/>